

FLOWSTATE — UPDATED IMPLEMENTATION GUIDE

Gurugram Street-wise Waterlogging Prediction, 3-hour Early Warning, Drainage
Intelligence and Flood-aware Routing

Prepared from the supplied SIH project and rebuilt as a hackathon-ready updated
architecture.

Purpose: explain exactly what was changed, how the new modules work, how to run the system, and what remains a
prototype limitation.

1. Executive Summary

The updated build moves the project from zone-level flood visualization to a street-segment prediction architecture. Each road segment can now receive a live risk category, flood probability, predicted water depth and a 0–180 minute forecast. Drain capacity and blockage are included as prediction inputs. The route engine uses a local road graph and Dijkstra shortest-path search so the “safe” mode can exclude HIGH and SEVERE road segments when a safe path exists.

The project is intentionally honest about data quality: the bundled ML model is a synthetic calibration model for demonstration. It must be retrained with labelled Gurugram observations before operational deployment.

The Gurugram elevation calibration was corrected to an approximately 190–280 m range rather than the earlier unrealistic 3–35 m values. Official/agency sources report Gurugram around 217–225 m average/elevation depending on the reference area. [Sources listed at the end.]

2. Major Updates

Area	Before	Updated build
Map	Large zone circles	Individual road-segment polylines with risk colours
Prediction	Zone-oriented nowcast	Road-level 0/30/60/90/120/150/180 minute forecast
ML	Synthetic RF with mismatched elevation calibration	RF classifier + depth regressor with realistic Gurugram elevation range
Drainage	Synthetic drain markers and zone connections	Drain assets with capacity, blockage, effective capacity and status
Routing	Detour-distance estimate	Road-graph Dijkstra with flood-risk edge filtering/penalties
Safe route	Approximate	Shortest route over safe graph when available
Admin	Demo role header	Signed admin token and protected actions
Weather	Open-Meteo UTC	Open-Meteo Asia/Kolkata, cached and clearly labelled fallback
Google Maps	Open directions link	Explicit navigation handoff after FlowState route calculation
OSM	Not integrated	Overpass import script included for real road geometry

3. New Architecture

Data flow: Open-Meteo → weather ingestion/cache → feature engineering → Random Forest + drainage/physics features → street risk → map/alerts/routing. Road geometry comes from the seeded Gurugram demo network or can be replaced by OpenStreetMap/Overpass data using the included import script.

Routing flow: road segments become graph edges. Each edge gets travel cost and flood risk. Safe mode removes HIGH/SEVERE edges when possible; balanced mode adds a risk penalty to travel time; normal mode minimizes travel time on the available graph.

Alert flow: the alert refresh operation evaluates 60, 120 and 180 minute horizons and creates active alerts for HIGH/SEVERE streets.

4. Street-wise Waterlogging Map

The map now draws every route segment as a coloured line. Green represents LOW, yellow MODERATE, orange HIGH and red SEVERE. Clicking a road opens its current prediction and a seven-point 3-hour timeline. Drain markers are separate and show blockage, effective capacity and status.

For a production/SIH field-data version, run `python scripts/import_osm.py` from the backend with internet access. The script queries an Overpass endpoint for Gurugram highway ways and stores them as route segments. The prediction engine then evaluates each imported segment.

Important: the included demo road network uses named Gurugram roads and representative geometry so the project works immediately without an external GIS import. It is not a substitute for authoritative municipal road geometry.

5. Prediction Model

The model uses 12 features: recent rainfall, forecast rainfall, elevation, slope, impervious surface, drainage utilization, blockage, historical risk and road factor. Two Random Forest models are trained: a classifier for waterlogging probability and a regressor for predicted water depth.

Feature group	Features
Rainfall	rain_1h, rain_3h, rain_6h, forecast_1h, forecast_3h
Terrain	elevation_m, slope_percent
Urban form	impervious_percent
Drainage	drainage_utilization, blockage_percent
History	historical_risk
Road context	road_factor

The included model is synthetic calibration only. Its metrics are shown in the training metadata, but they must not be described as real-world Gurugram accuracy. A proper deployment needs historical rainfall, observed waterlogging labels/depths, validated drainage data and DEM/land-use layers.

6. 3-hour Advanced Alert System

The alert engine evaluates +60, +120 and +180 minutes. A street becomes alert-worthy when its predicted category is HIGH or SEVERE. Each alert contains the road, severity, horizon and predicted depth. The admin interface exposes a refresh operation so the demo can update alerts on demand.

For continuous deployment, schedule the refresh operation every 5–10 minutes using Windows Task Scheduler, cron, Celery, APScheduler or a managed job runner. The current package deliberately keeps the implementation simple for a 24-hour hackathon.

7. Drainage System

Every drain stores design capacity and blockage. Effective capacity is calculated as $\text{normal capacity} \times (1 - \text{blockage}/100)$. The prediction engine converts rainfall/runoff and available drainage capacity into a utilization signal. Drain status is NORMAL, WARNING or CRITICAL. This provides a strong explainability layer: a road can be high-risk because rainfall is intense, drainage is overloaded, or blockage is high.

8. Safe Route / Shortest Safe Route

The previous implementation estimated a detour by adding a fixed distance per avoided segment. The updated implementation builds a graph from road endpoints and uses Dijkstra shortest-path search. In safe mode, HIGH/SEVERE segments are removed when a safe route is possible. The returned path is drawn on the map.

Three modes are available: (1) Safe — remove HIGH/SEVERE streets when possible; (2) Balanced — include flood risk as a route penalty; (3) Normal — shortest travel-time path on the graph.

If no fully safe route exists, the safe mode falls back to a balanced route so the system can still return a useful path. The UI explicitly states this behavior.

9. Open-Meteo Integration

The weather service requests hourly precipitation and probability plus current weather variables. The timezone is Asia/Kolkata. Responses are cached for ten minutes. If the provider is unreachable, the application returns a clearly labelled demo-fallback payload instead of pretending that synthetic values are live. Open-Meteo documentation states that the forecast API provides hourly forecasts and supports a timezone parameter. [Official source below.]

10. Google Maps Integration

Google Maps is used as a navigation handoff, not as the flood-risk engine. The user first receives a flood-aware route from FlowState, then can continue in Google Maps using a Maps Directions URL. Google's official Maps URLs documentation states that the directions endpoint can launch directions using origin, destination and travel mode, and that Maps URLs do not require a Google API key.

11. Admin Panel

The updated admin panel has a signed token-based login and protected endpoints. The authority can refresh 3-hour alerts and refresh the street model. Drain blockage updates are also protected by admin authentication at the API layer. Change the demo password in .env before deployment.

12. Running the Project

Backend: `cd backend → python -m venv venv → activate venv → pip install -r requirements.txt → copy .env.example .env → flask --app app init-db → flask --app app seed-db → python -m ml.train → python app.py`

Frontend: `cd frontend → npm install → npm run dev`

Real OSM: from backend, run `python scripts/import_osm.py` after database seeding. Internet access is required.

13. SIH Demo Sequence

- Open Gurugram map and show green/yellow/orange/red street segments.
- Click a red road and show probability, predicted depth and +0 to +3 hour forecast.
- Open a drain and show blockage/effective capacity.
- Refresh 3-hour alerts and show a HIGH/SEVERE warning.
- Select two points and demonstrate Safe versus Normal route.
- Click Continue in Google Maps to demonstrate navigation handoff.
- Explain that the model is a calibrated prototype and that operational deployment requires real labelled flood data.

14. Known Limitations and Next Steps

- The bundled ML model is synthetic and not operationally validated.
- The seeded road network is representative demo geometry; use OSM/municipal GIS for authoritative street geometry.
- The OSM importer now requests full OSM geometry and stores consecutive geometry pairs as graph edges, preserving road topology more faithfully for routing.
- Drain locations/capacities are demo infrastructure data until replaced by municipal GIS/asset data.
- A production alert scheduler, audit log, rate limiting and secret management should be added.
- For scientific validation, use held-out historical storms and report precision, recall, F1, calibration, depth MAE/RMSE and false-alarm rate.

15. Recommended Production Data

Data	Recommended source/use
Road network	OpenStreetMap/municipal GIS; preserve topology
Elevation/slope	Validated DEM; derive slope per road segment
Land cover	GIS/satellite-derived impervious fraction
Weather	Open-Meteo plus a validated local rainfall gauge/radar source where available
Drainage	Municipal/GMDA/MCG/HSVP asset inventory and capacity
Labels	Historical waterlogging reports, sensors, CCTV annotations, citizen reports
Routing	Local graph or a routing service; keep flood filtering in FlowState

16. Sources

Open-Meteo documentation: <https://open-meteo.com/en/docs>

Google Maps URLs documentation: <https://developers.google.com/maps/documentation/urls/get-started>

OSRM HTTP API documentation: <https://project-osrm.org/docs/v26.4.0/http>

Gurugram elevation/topography references: Haryana/agency documents cited during review; see the accompanying project report.

OpenStreetMap/Overpass documentation: https://wiki.openstreetmap.org/wiki/Overpass_API

Final note: This updated package is designed to be honest, demoable and extensible. It should be presented as a prototype decision-support system, not as a certified public-safety forecasting service.